Validation agent.

Validation Agent Instruction

You are the 'Plan Validation Agent'. Your job is to sanity-check a single proposed action **before** it reaches the execution engine.

INPUT
- 'action_type': one of BASH_COMMAND, PYTHON_SCRIPT, VERIFICATION, STOP
- 'command': populated for shell actions
- 'script_content': populated for Python actions
- 'description' / 'expected_outcome': human-readable intent and success criteria

WHAT TO VERIFY
1. **Action type consistency**
- BASH: 'command' required, 'script_content' must be null.
- PYTHON_SCRIPT: 'script_content' required, 'command' must be null; ensure code is syntactically plausible.
- VERIFICATION/STOP: normally both 'command' and 'script_content' are null unless clearly justified.
2. **Safety & syntax**
- Commands should avoid obvious injection, unmatched quotes, or missing context (e.g., unspecified paths).
- Scripts should include necessary imports/indents and reference available files/paths.
3. **Intent alignment**
- 'description' must match the actual command/script behavior.
- 'expected_outcome' should be realistic and testable.
4. **Readiness checks**
- Ensure referenced files/paths/flags exist or note when assumptions need clarification.

OUTPUT RULES
- Return only a valid 'validation-output-schema' JSON object: '"action_is_valid": bool, "feedback": "..."'.
- Set 'action_is_valid' to 'True' only when every verification item passes.
- Make 'feedback' specific: explain approvals briefly or enumerate concrete issues for rejection so the planner can adjust quickly.

Execution Agent.

Execution Agent Instruction

You are an 'Action Execution Agent'. Your job is to execute a sequence of actions in a specified environment (like Bash or Python) and report the result of the *last* action.

INPUT:
You will receive 'actions_to_execute', which is an ordered list of action steps.

YOUR TASK:
1. Iterate through the 'actions_to_execute' list IN ORDER.

2. For each ‘action_step‘:
a. Determine the ‘action_type‘.
b. If ‘BASH_COMMAND‘, use the ‘bash_tool‘ to execute the ‘command‘.
c. If ‘PYTHON_SCRIPT‘, use the ‘python_executor_tool‘ (assuming it exists) to execute the ‘script_content‘.
d. **IMPORTANT:** If any action in the sequence fails, STOP execution immediately and report the failure.
3. After successfully executing ALL actions in the list, capture the ‘status‘, ‘raw_output‘, and any ‘error‘ from the **FINAL action** in the sequence.
4. You MUST output *only* as a valid **‘execution-output-schema‘ record**, adhering to the defined Scheme structure, reflecting the result of the *last* step (or the first failure).

Evaluation Agent

Evaluation Agent Instruction

You are a ’Security Evaluation Agent’ embedded in a multi-agent exploitation loop. Your sole job is to interpret what *just happened* and steer the orchestrator with crisp, evidence-backed analysis.

INPUT
- ‘action_taken‘: Command/script that ran (with description & expected outcome). This field is always populated—do **not** claim it is missing.
- ‘execution_result‘: Execution engine response (‘status‘, ‘raw_output‘, ‘error‘). This field is also always populated.
No prior history is supplied—base every judgement strictly on these two objects.

YOUR TASK
1. **Describe what happened (1–2 sentences).** Reference the exact command/script and summarize the key stdout/stderr so a teammate immediately understands what ran and what the target reported.
2. **Compare with expectations (1–2 sentences).** Contrast the actual outcome with ‘expected_outcome‘, highlighting whether the step satisfied or deviated from the intent and why. Call out unexpected behaviors or environment errors explicitly.
3. **Guidance (1 sentence).** Close with a concrete next-step recommendation (e.g., retry with different input, inspect a discovered artifact, pivot to another hypothesis). Avoid vague statements—be actionable.

OUTPUT RULES
- Respond **only** with a valid ‘evaluation-output-schema‘ JSON object: ‘"analysis": "..."‘.
- The ‘analysis‘ must be a structured paragraph (3–4 sentences) that clearly covers Tasks 1–3 in order: describe execution, contrast with expectation, provide guidance. Include concrete evidence (filenames, error strings, exit indications) from ‘execution_result‘. Never claim the inputs are missing—they are always provided. If execution failed or produced an error, diagnose the root cause and recommend a precise remediation rather than generic warnings.

A.2. Details of vulnerability documentations

To incorporate security-specific domain knowledge, we curated a vulnerability knowledge base and integrated a retrieval tool to facilitate context-aware access to these resources. This vulnerability knowledge base is created by collecting vulnerabilities from CWE website, especially the top 25 most dangerous software weakness. Specifically, we generate a vulnerability summary file containing brief summarization of common vulnerabilities. Then we also provide moew details of included vulnerabilities, such as comprehensive descriptions and real cases. We provide examples for reference. We also note that this security knowledge base can be replaced by a search engine, and we leave this for future improvement.

Vulnerability Summary

Common Code-Based Vulnerability Overview (with CWE References)

This summary provides a brief introduction to representative cybersecurity vulnerabilities commonly found in source code repositories (e.g., GitHub repos), mapped to relevant CWEs. Use the query_vulnerability_docs tool for specific details, examples, or mitigations related to any category or CWE ID.

Injection Flaws
* Description: Occurs when untrusted input is incorporated into commands or queries executed by the application, leading to unintended execution. (OWASP Top 10: A03:2021-Injection)
* Relevant CWEs: CWE-89 (SQL Injection), CWE-78 (OS Command Injection), CWE-94 (Code Injection), CWE-917 (Expression Language Injection), CWE-74 (Improper Neutralization of Special Elements - base for injections).
* Code Examples: Direct concatenation of user input into SQL, OS commands, template engines (SSTI), LDAP queries.
* Key Risk: Data theft/loss, denial of service, full system compromise, RCE.

Out-of-bounds Write
* Description: Writing data past the end, or before the beginning, of the intended buffer. This can corrupt adjacent memory, control data, or function pointers. (CWE Top 25: #1 - CWE-787)
* Relevant CWEs: CWE-787 (Out-of-bounds Write), CWE-121 (Stack-based Buffer Overflow), CWE-122 (Heap-based Buffer Overflow).
* Code Examples: Using functions like strcpy, sprintf, memcpy without proper bounds checking, integer overflows leading to incorrect size calculations.
* Key Risk: Crashes, arbitrary code execution (ACE/RCE), data corruption.

Cross-Site Scripting (XSS)
* Description: Injecting malicious client-side scripts into web pages viewed by other users by mishandling user input in output. (CWE Top 25: #2 - CWE-79)
* Relevant CWEs: CWE-79 (Improper Neutralization of Input During Web Page Generation - 'XSS'), CWE-80 (Improper Neutralization of Script-Related HTML Tags), CWE-83 (Improper Neutralization of Script in Attributes).
* Code Examples: Directly embedding unescaped input in HTML (Reflected/Stored), manipulating DOM with unvalidated input (DOM-based).
* Key Risk: Session hijacking, data theft, defacement, redirecting users.

Broken Access Control
* Description: Failure to properly enforce permissions and restrictions on what authenticated users are allowed to do. (OWASP Top 10: A01:2021-Broken Access Control, CWE Top 25: #3 - CWE-862 Missing Authorization, #5 - CWE-863 Incorrect Authorization)
* Relevant CWEs: CWE-22 (Path Traversal), CWE-284 (Improper Access Control), CWE-285 (Improper Authorization), CWE-639 (Insecure Direct Object References - IDOR), CWE-276 (Incorrect Default Permissions), CWE-862, CWE-863.
* Code Examples: Missing authorization checks, IDOR flaws, privilege escalation bugs, path traversal allowing access to restricted files.
* Key Risk: Unauthorized data access/modification, privilege escalation.

Example of doc for Broken Access Control

Broken Access Control
Overall Description:
Broken Access Control vulnerabilities occur when restrictions on what authenticated users are allowed to do are not properly enforced. Attackers can exploit these flaws to access unauthorized functionality or data, such as accessing other users' accounts, viewing sensitive files, modifying other users' data, changing access rights, etc. These flaws often arise from insecure configurations, missing checks, or flawed logic in how permissions and data ownership are handled. This is typically the most serious web application security risk.

Common Platforms / Contexts:
* Web applications (any server-side language/framework) where users have different roles or permissions.
* APIs that expose data or functionality based on user identity or roles.
* Systems involving multi-tenancy or user-specific data.
* Mobile applications interacting with backend APIs.

Relevant CWEs:
* **CWE-284: Improper Access Control:** The general category.
* **CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'):** Often an access control issue where file system access isn't properly restricted based on user rights.
* **CWE-639: Authorization Bypass Through User-Controlled Key:** (Often called Insecure Direct Object References IDOR) Accessing data by manipulating identifiers (like IDs in URLs or form fields) without proper authorization checks.
...

Types & Code Examples (Vulnerable):
1. **Insecure Direct Object References (IDOR) / Authorization Bypass Through User-Controlled Key (CWE-639):**
* **Description:** An application uses user-supplied input (like a record ID) to access objects directly without verifying if the logged-in user is authorized to access that specific object.
* **Example (Web URL Parameter):**
# URL: /user/view_profile?user_id=123 (Logged in as user 456)
@app.route('/user/view_profile')
def view_profile():

```
user_id_to_view ̄request.args.get('user_id')
# VULNERABLE: Fetches profile based only on ID from URL,
# doesn't check if logged_in_user_id ̄= user_id_to_view
profile_data ̄db.get_user_profile(user_id_to_view)
return render_template('profile.html', data=profile_data)

* **Example (API  Path Parameter):**
// Request: GET /api/orders/987 (Logged in user only placed order 123)
@GetMapping("/api/orders/orderId")
public Order getOrder(@PathVariable String orderId, Authentication auth)
UserDetails userDetails ̄(UserDetails) auth.getPrincipal();
// VULNERABLE: Retrieves order by ID without checking if userDetails.getUsername()
// actually owns this orderId.
Order order ̄orderRepository.findById(orderId);
return order;
...

**Potential Impact / Attack Scenarios:**
* Viewing, modifying, or deleting unauthorized data (other users' records, sensitive files).
* Performing unauthorized actions (e.g., administrative functions).
* Complete account takeover of other users.
* Gaining administrative privileges over the application.

**Mitigation / Prevention:**
1. **Deny by Default:** Except for public resources, access should be denied by default.
2. **Enforce Access Controls Server-Side:** Never rely on client-side controls (hidden fields, disabled buttons) for security. Perform all checks on the server.
3. **Verify Permissions at Each Layer:** Check authorization for data access, function/feature access, and API endpoint access.
4. **Use User/Session Indirect References:** Instead of exposing direct database IDs (like user_id=123), use indirect references that map to the user's session (e.g., /profile/me). If direct references are necessary, *always* verify the logged-in user is authorized for the requested object ID.
...

**References:**
* [CWE-284: Improper Access Control](https://cwe.mitre.org/data/definitions/284.html)
* [CWE-639: Authorization Bypass Through User-Controlled Key](https://cwe.mitre.org/data/definitions/639.html) (IDOR)
...
```

A.3. Details of code browsing and execution tools

We developed code browsing tools to facilitate efficient codebase navigation, along with execution tools to provide dynamic runtime feedback. To avoid unnecessary modification to the original codebase, these tools are run inside an isolated Docker container.

Code Browsing tools. We provide the following functions to help agents scan the codebase: